

بخش 1: Serializer

1) ثبت‌نام کاربر (Serializer محور)

سناریو

می‌خواهیم یک endpoint برای ثبت‌نام کاربر داشته باشیم.

خواسته‌ها

- استفاده از `Serializer` (نه `ModelSerializer`)
- اعتبارسنجی:
 - یکتا بودن email
 - حداقل طول پسورد
- هش کردن پسورد
- عدم نمایش پسورد در response

مدل‌های موردنیاز

```
from django.contrib.auth.models import User
```

Hint

- از `serializers.Serializer` استفاده کن.
- برای هش کردن پسورد از:

```
(user.set_password(password
```

استفاده کن.

- برای مخفی کردن password در response می‌توانی از:

```
write_only=True
```

استفاده کنی.

- اعتبارسنجی ایمیل را داخل:

validate_email

پیاپیادهسازی کن.

مفاهیم

validate, validate_<field>, create, security

(2) پروفایل کاربر

سناریو

هر کاربر یک پروفایل دارد (شماره تماس، بیو، تصویر).

مدل‌های موردنیاز

```
from django.contrib.auth.models import User
```

```
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    phone = models.CharField(max_length=20)
    bio = models.TextField(blank=True)
    avatar = models.ImageField(upload_to="avatars")
```

خواسته‌ها

- استفاده از `ModelSerializer`
- `nested serializer` برای نمایش اطلاعات کاربر
- امکان `update` فقط برای صاحب پروفایل

Hint

- برای نمایش اطلاعات کاربر:

```
(user = UserSerializer(read_only=True
```

- از `permission` سفارشی استفاده کن.
- `read_only_fields` را برای `user` قرار بده.

مفاهیم

nested serializer، `read_only_fields`

3) ساخت خودکار پروفایل بعد از ایجاد کاربر

سناریو

بعد از ثبت `User`، به صورت خودکار یک `Profile` ساخته شود.

مدل‌های موردنیاز

```
from django.contrib.auth.models import User
```

```
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
```

خواسته‌ها

- استفاده از `post_save`
- ساخت خودکار `Profile`
- فقط هنگام `create` شدن `user` اجرا شود

Hint

- از `signal` زیر استفاده کن:

```
from django.db.models.signals import post_save
```

- بررسی کن:

```
:if created
```

- `signal` را داخل فایل:

```
signals.py
```

قرار بده.

- فراموش نکن `apps.py` را تنظیم کنی.

مفاهیم

`post_save`, signals, app config

بخش 2: Authentication & Permission

4) فقط صاحب منبع

سناریو

کاربر فقط می‌تواند پست‌های خودش را ویرایش یا حذف کند.

مدل‌های موردنیاز

```
class Post(models.Model):
    owner = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=255)
    content = models.TextField
```

خواسته‌ها

- پیاده‌سازی `permission` سفارشی
- اعمال `permission` روی `ViewSet`

Hint

- از:

`BasePermission`

استفاده کن.

- داخل:

`has_object_permission`

مالک را چک کن.

مفاهیم

BasePermission, has_object_permission

(5) نقش‌ها (Role-based Access)

سناریو

کاربران نقش دارند:

- admin
- staff
- user

مدل‌های موردنیاز

```
:(class UserRole(models.TextChoices)
    "ADMIN = "admin
    "STAFF = "staff
    "USER = "user
```

```
:(class Profile(models.Model)
    (user = models.OneToOneField(User, on_delete=models.CASCADE)
    (role = models.CharField(max_length=20, choices=UserRole.choices)
```

خواسته‌ها

- فقط admin اجازه delete دارد
- staff اجازه update دارد
- user فقط read

Hint

- method-based permission پیاده‌سازی کن.
- `SAFE_METHODS` را بررسی کن.

مفاهیم

بخش 3: View / ViewSet / Router

ViewSet کامل محصولات (6)

سناریو

مدیریت محصولات فروشگاه

مدل‌های موردنیاز

```
(class Product(models.Model):
    owner = models.ForeignKey(User, on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    is_active = models.BooleanField(default=True)
```

خواسته‌ها

- استفاده از `ModelViewSet`
- `override` کردن:
 - `get_queryset`
 - `perform_create`
- فیلتر محصولات فعال برای کاربران عادی

Hint

- داخل `perform_create`:
`(serializer.save(owner=self.request.user`

- در `get_queryset` نقش کاربر را بررسی کن.

مفاهیم

ViewSet lifecycle

endpoint سفارشی (custom action) (7)

سناریو

برای هر محصول، یک endpoint داشته باشیم:

/POST /products/{id}/like

مدل‌های موردنیاز

```
(class ProductLike(models.Model)
    (user = models.ForeignKey(User, on_delete=models.CASCADE)
    (product = models.ForeignKey(Product, on_delete=models.CASCADE)
```

```
:class Meta
    ("unique_together = ("user", "product
```

خواسته‌ها

- استفاده از `action@`
- جلوگیری از like تکراری

Hint

- از:

```
(["action(detail=True, methods=["post@
```

استفاده کن.

- قبل از `create` بررسی کن که رکورد وجود نداشته باشد.

مفاهیم

(`action(detail=True@`

8) لیست و ساخت دسته‌بندی با GenericAPIView

سناریو

می‌خواهیم API برای لیست و ساخت category داشته باشیم.

مدل‌های موردنیاز

```
(class Category(models.Model):  
    name = models.CharField(max_length=255
```

خواسته‌ها

- استفاده از:
 - `ListCreateAPIView`
- فقط admin بتواند create کند

Hint

- از:
`generics.ListCreateAPIView`

استفاده کن.

- permission را بر اساس method کنترل کن.

مفاهیم

Generic Views، reusable views

9) جزئیات محصول با RetrieveAPIView

سناریو

نمایش جزئیات یک محصول

خواسته‌ها

- استفاده از `RetrieveAPIView`

- برگرداندن 404 در صورت نبود محصول

Hint

- فقط `serializer_class` و `queryset` کافی است.

مفاهیم

RetrieveAPIView

10) حذف محصول با DestroyAPIView

سناریو

فقط admin بتواند محصول را حذف کند.

خواسته‌ها

- استفاده از `DestroyAPIView`

Hint

- از `permission` سفارشی استفاده کن.

مفاهیم

DestroyAPIView

بخش 4: Filtering / Pagination / Search 🔍

11) لیست سفارش‌ها

سناریو

لیست سفارش‌های کاربر با قابلیت:

- pagination

- فیلتر بر اساس status
- sort بر اساس تاریخ

مدل‌های موردنیاز

```

class Order(models.Model):
    status = STATUS_CHOICES
    created_at = models.DateTimeField(auto_now_add=True)
    user = models.ForeignKey(User, on_delete=models.CASCADE)

    ("pending", "Pending")
    ("paid", "Paid")
    (
        status = models.CharField(max_length=20, choices=STATUS_CHOICES)
        (created_at = models.DateTimeField(auto_now_add=True)
        (user = models.ForeignKey(User, on_delete=models.CASCADE)

```

Hint

- از:

DjangoFilterBackend

و:

OrderingFilter

استفاده کن.

مفاهیم

[django-filter](#), [OrderingFilter](#), [PageNumberPagination](#)

12) جستجوی پیشرفته

سناریو

جستجو در محصولات بر اساس:

- نام
- توضیح
- بازه قیمت

Hint

- custom FilterSet بساز.
- از:

min_price
max_price

در query params استفاده کن.

مفاهیم

custom filterset، query params

بخش 5: Validation & Business Logic

13) جلوگیری از خرید بیش از موجودی

مدل‌های موردنیاز

```
:(class Product(models.Model  
()stock = models.PositiveIntegerField
```

خواسته‌ها

- validation در serializer
- atomic transaction

Hint

- اعتبارسنجی تعداد را داخل serializer انجام بده.
- از:

transaction.atomic

استفاده کن.

مفاهیم

`validate, transaction.atomic`

14) جلوگیری از ارسال درخواست تکراری

سناریو

کاربر نباید بتواند در ۵ ثانیه چند سفارش یکسان ثبت کند.

Hint

- از `cache` استفاده کن.
- `key` مناسب برای `user` و `payload` بساز.

مفاهیم

`throttling`، `cache`، `custom throttle`

بخش 6: Testing در DRF

15) تست API ثبت سفارش

خواسته‌ها

- تست `status code`
- تست ایجاد رکورد در دیتابیس
- تست `permission`

Hint

- از:

`APITestCase`

استفاده کن.

- تعداد رکوردها را قبل و بعد بررسی کن.

تست Permission (16)

سناریو

- کاربر معمولی نمی‌تواند delete کند
- admin می‌تواند

Hint

- از:

force_authenticate

استفاده کن.

مفاهیم

auth in tests، force_authenticate

بخش 7: سناریوهای واقعی و چالشی

bulk create (17)

سناریو

آپلود لیست محصولات به صورت bulk

خواسته‌ها

- دریافت لیست JSON
- validate هر آیتم
- ذخیره بهینه

Hint

- از:

`many=True`

استفاده کن.

- برای `performance` از:

`bulk_create`

استفاده کن.

مفاهیم

`many=True`, `bulk_create`

soft delete (18)

سناریو

حذف منطقی رکوردها

مدل‌های موردنیاز

```
:(class Product(models.Model)
(is_deleted = models.BooleanField(default=False
```

خواسته‌ها

- `override queryset`
- عدم نمایش رکوردهای حذف‌شده
- `endpoint restore`

Hint

- رکورد را `delete` واقعی نکن.
- `custom action` برای `restore` بساز.

مفاهیم

queryset management، custom action

versioning API (19)

سناریو

نسخه v1 و v2 برای محصولات داریم.

خواسته‌ها

- پشتیبانی از versioning

Hint

- از:

URLPathVersioning

یا:

NamespaceVersioning

استفاده کن.

مفاهیم

API Versioning

تمرین نهایی (جمع‌بندی) 

mini API فروشگاه (20)

پیاده‌سازی API شامل:

- (Auth (login/register
- Product
- Cart
- Order
- Permission
- Pagination
- Filtering
- Tests

خواسته‌ها

- معماری تمیز
- status code استاندارد
- validation مناسب
- business logic جدا از view

Hint

- logic سنگین را داخل service layer منتقل کن.
- serializer فقط validation انجام دهد.
- permission ها را reusable بنویس.

تمرکز روی

- تمیز بودن API
- استاندارد بودن status code
- خطاهای قابل فهم
- جدا بودن business logic